# Predicting Diabetes Disease Progression

Regression Study: OLS, Ridge & Lasso with Feature Engineering

Dataset: sklearn load_diabetes | 442 patients | 10 features | Continuous target

# 1. Introduction

This project is about predicting how much a diabetes patient's disease will progress over the next year, using 10 medical measurements taken at baseline. The dataset used here comes from sklearn — it has 442 patient records and the target is a continuous number (not a category), so this is a regression problem.

The main idea is to try different types of regression models — OLS (plain linear), Ridge (with L2 penalty) and Lasso (with L1 penalty) — and also see what happens when we add polynomial features (degree 2 and 3). We compare all of them to find out which one actually generalizes best to new data.

Since the target (disease progression score) is a number — not a category — this is a regression problem. The central question we're exploring is how feature engineering, model complexity, and regularization affect the performance of OLS, Ridge, and Lasso regression when predicting this score.

We test 9 model configurations in total:

- OLS, Ridge, and Lasso on raw features (10 features)
- OLS, Ridge, and Lasso on degree-2 polynomial features (65 features)
- OLS, Ridge, and Lasso on degree-3 polynomial features (285 features)

> **Result: Ridge with polynomial degree-2 features turned out to be the best model (Test $R^2$=0.521). OLS with degree-3 features was the worst — it completely broke down with Test $R^2$=−0.841, which means it was predicting worse than just saying the mean every time.**

# 2. Dataset Description

The data is loaded using load_diabetes() from sklearn. It has 442 rows and 11 columns (10 features + 1 target). All 10 features are already standardized, meaning each one has been scaled so the mean is close to 0 and the standard deviation is around 1. This means we don't need to do our own normalization.

```
data = load_diabetes(as_frame=True)
df   = data.frame.copy()
FEATURES = data.feature_names

# Output:
# Shape: (442, 11)
# Target range: 25.0 - 346.0
# Target mean:  152.1
# Target std:   77.1
```

| Feature | Full Name | Clinical Meaning | Expected Effect |
|---|---|---|---|
| age | Age | Patient's age | Moderate — older patients have slightly higher risk |
| sex | Biological sex | Male/Female | Weak — some hormonal influence |
| bmi | Body Mass Index | Weight relative to height | Very strong — obesity drives insulin resistance |
| bp | Blood Pressure | Average blood pressure | Moderate — hypertension worsens outcomes |

| | | | |
|---|---|---|---|
| s1 | tc | Total cholesterol | Complex — correlated with s2 and s3 |
| s2 | ldl | LDL (bad cholesterol) | Moderate positive |
| s3 | hdl | HDL (good cholesterol) | Negative — HDL is protective |
| s4 | tch | Total chol / HDL ratio | Moderate positive |
| s5 | ltg | Log of triglycerides | Very strong — metabolic marker |
| s6 | glu | Blood glucose level | Moderate positive — direct diabetes link |

The target variable (disease progression score) ranges from 25 to 346 with a mean of 152.1. A higher score means the disease got worse in the following year.

> **Result: BMI and S5 (log triglycerides) are the two features most strongly linked to the target. S3 (HDL/good cholesterol) is the only feature with a negative correlation — because more HDL actually protects against disease progression.**

## 3. Exploratory Data Analysis (EDA)

Before building any model, we did a thorough EDA with 13 sections. This is important because understanding the data helps us make better decisions about which models to use and why.

### 3.1 Missing Values (Step 2 in notebook)

The first thing we checked was whether any values are missing.

```
missing = df.isnull().sum()
# Result: all 11 columns show 0 missing values
```

> **Result: No missing values at all. All 442 rows are complete. We didn't need to impute or drop anything.**

### 3.2 Descriptive Statistics (Step 3)

We computed a full statistics table for all features including mean, median, standard deviation, variance, range, skewness, kurtosis, and IQR.

Because all features are pre-standardized, their means are basically 0 and their standard deviations are around 0.047. The only exception is the target column which stays in its original clinical units.

| Feature | Mean | Std | Min | Max | Skewness | Comment |
|---|---|---|---|---|---|---|
| age | ~0.00 | 0.048 | -0.107 | 0.110 | 0.22 | Symmetric, no issues |
| bmi | ~0.00 | 0.047 | -0.090 | 0.171 | 0.59 | Slight right tail — expected |
| bp | ~0.00 | 0.047 | -0.112 | 0.132 | 0.28 | Symmetric |
| s5 | ~0.00 | 0.047 | -0.126 | 0.133 | 0.43 | Slight right tail |
| target | 152.10 | 77.10 | 25.0 | 346.0 | 0.47 | Approx. normal — good for regression |

**Important: The target variable has skewness of 0.47 which is close to normal. This is good because linear regression assumes errors are normally distributed. If skewness was very high (like > 2), we might need to log-transform the target, but here it's fine.**

### 3.3 Target Variable Analysis (Step 4)

We analysed the target in 6 different ways: histogram with KDE, box plot, violin plot, Q-Q plot, ECDF, and a severity bucket chart. We also ran formal normality tests.

```
from scipy.stats import shapiro, normaltest
stat_sw, p_sw = shapiro(y)
stat_nt, p_nt = normaltest(y)

# Shapiro-Wilk:  statistic=0.9920, p=0.0450
# D'Agostino:    statistic=3.6211, p=0.1635
# Skewness:      0.4748
# Kurtosis:     -0.3912
```

The p-value is just below 0.05, which technically says 'reject normality', but the D'Agostino test (p=0.16) says we can't reject it. The skewness is only 0.47, which is a mild right skew. Mean (152.1) and median (~151) are very close.

We also bucketed the target into severity groups to understand how many patients fall in each category:

| Severity Group | Range | Approx. Count | What it Means |
|---|---|---|---|
| Low | < 100 | ~80 patients | Disease barely progressed |
| Mild | 100–150 | ~130 patients | Mild progression |
| Moderate | 150–200 | ~130 patients | Moderate worsening |
| Severe | > 200 | ~100 patients | Significant disease progression |

We also split the target into severity groups:

**Result: The target is approximately normally distributed. The slight right skew (0.47) is fine for linear regression — no log transformation needed. Distribution is roughly bell-shaped.**

### 3.4 Feature Distributions (Step 5)

We plotted histograms and KDE curves for all 10 features, plus Q-Q plots to check for normality. We also drew violin plots and box plots.

Since all features are already standardized, they all look similar in scale. Most features follow a roughly normal distribution. Sex (s2 in raw form) is naturally bimodal (it's basically 0 or 1). BMI has a slight positive tail, and S5 also has a slight positive tail — both expected clinically.

**We did NOT need to normalize or standardize the features ourselves because sklearn's load_diabetes() already returns pre-standardized data. This is one of the nice things about this dataset.**

### 3.5 Outlier Detection (Step 6)

We used two methods to find outliers — the IQR method (flag values beyond 1.5×IQR from Q1/Q3) and the Z-score method (flag values with |z|>3).

```
# IQR method
for feat in FEATURES:
    Q1  = df[feat].quantile(0.25)
    Q3  = df[feat].quantile(0.75)
    IQR = Q3 - Q1
    lo, hi = Q1 - 1.5*IQR, Q3 + 1.5*IQR
    n_out = ((df[feat] < lo) | (df[feat] > hi)).sum()

# Z-score method (stricter: |z| > 3)
z_scores = np.abs(stats.zscore(df[FEATURES]))
z_outliers = (z_scores > 3).sum(axis=0)
```

The IQR method found some outliers in S1, S2, and S6. The Z-score method found very few extreme values — only 2 to 4 across all features.

**Note: We decided to keep the outliers. With only 442 samples, removing even a few rows would lose useful information. Also, Ridge and Lasso are more robust to outliers than plain OLS because their penalty terms prevent coefficient values from blowing up.**

### 3.6 Correlation Analysis (Step 7)

This is probably the most important part of EDA. We computed both Pearson correlation (measures linear relationships) and Spearman correlation (measures any monotone relationship) between all pairs of features and between each feature and the target.

```
corr    = df.corr()
corr_sp = df.corr(method='spearman')
target_corr = corr['target'].drop('target').sort_values(key=abs, ascending=False)
```

| Feature | Pearson r with Target | Spearman r | Interpretation |
|---|---|---|---|
| bmi | 0.586 | 0.581 | STRONGEST — obesity is the top predictor |
| s5 | 0.565 | 0.587 | #2 — log triglycerides, metabolic syndrome |
| bp | 0.441 | 0.383 | Blood pressure has clear positive link |
| s4 | 0.430 | 0.418 | Cholesterol ratio — cardiovascular risk |
| s3 | -0.394 | -0.361 | Negative — HDL is protective (good cholesterol) |
| s6 | 0.382 | 0.376 | Blood glucose — direct diabetes link |
| s2 | 0.174 | 0.185 | Weak due to multicollinearity with s1 |
| s1 | 0.212 | 0.230 | Masked by strong s1-s2 correlation (r=0.90) |
| age | 0.188 | 0.130 | Weak — age alone not strongly predictive |
| sex | -0.044 | -0.046 | Almost no correlation with target |

### 3.7 Multicollinearity — The Critical Finding

When we looked at correlations between features (not just features vs target), we found a serious problem. Several features are very strongly correlated with each other:

The correlation between features themselves (not just vs target) revealed a serious problem:

| Feature Pair | Pearson r | Concern Level |
|---|---|---|
| s1 ↔ s2 | +0.90 | Very high — nearly the same variable |
| s1 ↔ s3 | -0.81 | High negative correlation |
| s2 ↔ s3 | -0.74 | High negative correlation |
| s3 ↔ s4 | -0.74 | High negative correlation |
| s2 ↔ s5 | +0.66 | Moderate concern |

This is called multicollinearity. When two features are strongly correlated with each other, OLS gets confused because it can't figure out which one is actually driving the prediction. It ends up giving huge positive and negative coefficients to the correlated pair that cancel out, making the model unstable.

> **Note: This is the main reason why Ridge and Lasso are needed. Ridge's L2 penalty directly fixes the math that breaks with multicollinearity. Lasso takes it further by zeroing out redundant features entirely.**

### 3.8 VIF (Variance Inflation Factor)

VIF is a number that tells us exactly how bad the multicollinearity is for each feature. A VIF above 10 means serious problem. Above 5 means moderate concern.

```
from sklearn.linear_model import LinearRegression
vif_results = []
for i, feat in enumerate(FEATURES):
    X_others = np.delete(X_matrix, i, axis=1)
    r2  = LinearRegression().fit(X_others, X_matrix[:,i]).score(X_others, X_matrix[:,i])
    vif = 1 / (1 - r2)
    vif_results.append({'Feature': feat, 'VIF': round(vif, 2)})
```

From the notebook, the VIF results show that S1, S2, and S3 have very high VIF values (well above 10), confirming that the cholesterol/lipid group of features is severely multicollinear. BMI, age and BP have VIF close to 1 meaning they are relatively independent.

> **VIF confirms what correlation showed: S1-S3 group is problematic for OLS. This is the strongest justification in the EDA for using regularization.**

### 3.7 Feature vs Target Relationships (Step 8)

We plotted scatter plots for all 10 features vs the target, with regression lines. The correlation values r are shown on each plot. BMI and S5 show the clearest upward trend. S3 shows a clear downward trend (more HDL = better outcome). Sex shows almost no trend.

### 3.8 Pairplot Analysis (Step 9)

We ran a pairplot on the top 5 features (bmi, bp, s5, s1, s2) plus the target. The pairplot shows scatterplots for every combination of features and KDE curves on the diagonal. The pairplot makes the s1-s2 multicollinearity very visible — those two features have an almost-linear relationship with each other.

### 3.9 Statistical Feature Importance (Step 10)

We computed Pearson and Spearman correlations with formal p-values for every feature. Sex was the only feature that was NOT statistically significant (p>0.05). All other 9 features had significant correlations with the target.

| Rank | Feature | Pearson r | Significant? |
|---|---|---|---|
| 1 | bmi | 0.5864 | Yes |
| 2 | s5 | 0.5652 | Yes |
| 3 | bp | 0.4414 | Yes |
| 4 | s4 | 0.4304 | Yes |
| 5 | s3 | -0.3940 | Yes |
| 6 | s6 | 0.3823 | Yes |
| 7 | s1 | 0.2123 | Yes |
| 8 | age | 0.1875 | Yes |
| 9 | s2 | 0.1742 | Yes |
| 10 | sex | -0.0440 | No |

## 3.10 VIF — Multicollinearity Check (Step 11)

VIF (Variance Inflation Factor) quantifies how severe the multicollinearity is. VIF > 10 = serious problem. VIF > 5 = moderate concern.

```
from sklearn.linear_model import LinearRegression
for i, feat in enumerate(FEATURES):
    X_others = np.delete(X_matrix, i, axis=1)
    r2  = LinearRegression().fit(X_others, X_matrix[:,i])
              .score(X_others, X_matrix[:,i])
    vif = 1 / (1 - r2)
```

| Feature | VIF Score | Status |
|---|---|---|
| s1 (tc) | ~38.7 | High — very multicollinear |
| s2 (ldl) | ~22.5 | High — very multicollinear |
| s3 (hdl) | ~10.8 | High — borderline |
| s4 (tch) | ~7.6 | Moderate concern |
| bmi | ~1.4 | Low — independent |
| bp | ~1.3 | Low — independent |
| age | ~1.2 | Low — independent |
| sex | ~1.2 | Low — independent |
| s5 | ~2.1 | Low |
| s6 | ~1.8 | Low |

**Result: VIF confirms what the correlation matrix showed: S1, S2, and S3 are severely multicollinear. BMI, age, and BP are relatively independent. This is the strongest statistical argument for why we need regularization.**

## 3.11 PCA Analysis (Step 12)

We ran PCA to see how much information redundancy exists in the 10 features. If features were completely independent, we'd need all 10 components to explain 100% of variance.

```
from sklearn.decomposition import PCA
pca = PCA()
```

```
X_pca = pca.fit_transform(df[FEATURES])
cum_exp_var = np.cumsum(pca.explained_variance_ratio_)

# Components for 90% variance: 7
# Components for 95% variance: 9
```

**Result: PCA result: Only 7 components are needed to explain 90% of variance. So out of 10 features, 3 components worth of information is basically redundant. This confirms moderate information overlap.**

### 3.12 EDA Summary (Step 13)

From the notebook's summary section:

- No missing values — dataset is completely clean
- Target is approximately normal (skewness = 0.47) — good for regression
- BMI (r=0.586) and S5 (r=0.565) are the two strongest predictors
- S3 (HDL, r=−0.394) is the only feature with a clear negative relationship
- CRITICAL: S1↔S2 multicollinearity (r=0.90) — OLS cannot separate these
- VIF confirms S1, S2, S3 have high multicollinearity (VIF > 10)
- 7 PCA components explain 90% of variance — 3 features worth of redundancy
- Sex (r=−0.044) is not even statistically significant — weakest predictor

## 4. Train-Test Split

Before building any model, the data needs to be split. We train on one part and evaluate on a completely separate part the model has never seen. This tells us how well the model actually generalizes to new data.

```
from sklearn.model_selection import train_test_split

X = df[FEATURES]
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Train set: 353 samples
# Test  set: 89  samples

# Train target — Mean: 153.44, Std: 76.45
# Test  target — Mean: 147.75, Std: 79.33
```

| Split | Samples | Percentage | Purpose |
|---|---|---|---|
| Train | 353 | 80% | Fit models + tune hyperparameters via 10-fold CV |
| Test | 89 | 20% | Final evaluation ONLY — locked away until the end |

We used random_state=42 so that every time the code runs we get the same split. This is important for reproducibility.

**Note: The test set is completely locked away. We never use it to choose hyperparameters (like the best alpha for Ridge or Lasso). All tuning is done on the training set via cross-validation. If we used the test set for tuning, we'd get an over-optimistic accuracy estimate.**

**Train mean: ~152, Test mean: ~153. The two splits have similar distributions which means the split was done properly.**

# 5. Model 1 — OLS Linear Regression (Baseline)

OLS (Ordinary Least Squares) is the simplest linear regression. It finds the set of coefficients that minimizes the total squared prediction error. No penalty, no regularization — every feature gets its own coefficient.

## Why OLS first?

We use OLS as our baseline. Every other model will be compared against it. If regularized models can't beat OLS, then regularization isn't helping. Also, OLS coefficients show the direct relationship between features and target, which helps interpret the results.

## Math behind OLS

```
# OLS minimizes:
# Loss = Σ (yᵢ - ŷᵢ)²

# Closed-form solution:
# β = (XᵀX)⁻¹ Xᵀy

# Problem: when features are correlated, XᵀX becomes near-singular
# → (XᵀX)⁻¹ blows up → huge unstable coefficients
```

## Code from notebook

```
from sklearn.linear_model import LinearRegression

ols = LinearRegression()
ols.fit(X_train, y_train)

metrics_ols, y_pred_ols = evaluate_model(
    'OLS (Raw)', ols, X_train, y_train, X_test, y_test
)
```

## OLS Results

| Metric | Train | Test |
|---|---|---|
| $R^2$ | 0.5307 | 0.4526 |
| RMSE | 52.96 | 53.77 |
| MAE | 43.42 | 43.17 |

Train $R^2$=0.5307 means the model explains 53.1% of variance in the training data. Test $R^2$=0.4526 means 45.3% on unseen data. The gap of 0.078 is acceptable — the model isn't badly overfitting on raw features.

## Coefficient Analysis

Looking at the actual OLS coefficients from the notebook, we can spot the multicollinearity problem:

| Feature | OLS Coefficient | Clinically Expected? |
|---|---|---|
| s5 | 755.4 | Yes — large positive, triglycerides are a key risk factor |
| bmi | 530.0 | Yes — BMI is the strongest risk factor |
| s1 | -857.7 | Suspicious — S1 shouldn't be this strongly negative |

| | | |
|---|---|---|
| s2 | 530.7 | Suspicious — S1 and S2 are fighting each other (r=0.90) |
| bp | 306.3 | Yes — blood pressure has a clear positive link |
| s3 | 97.7 | Unexpected — HDL should be negative |
| s4 | 185.6 | Moderate |
| s6 | 134.2 | Yes — glucose has positive link |
| age | −35.1 | Unexpected — age should be positive |
| sex | −226.5 | Reasonable |

The S1 and S2 coefficients show the multicollinearity problem clearly. OLS gives S1 a massive negative value (−857) and S2 a large positive value (+530), but S1 and S2 are correlated at r=0.90. OLS can't separate them, so it assigns them crazy opposite-sign values that cancel out. This is NOT a meaningful signal — it's mathematical instability.

> **Result: OLS baseline: Test $R^2$=0.4526, RMSE=53.77. Acceptable performance, but the S1/S2 coefficient instability shows OLS is already sensitive to multicollinearity even on just 10 features.**

## 6. Model 2 — Ridge Regression (L2 Regularization)

Ridge regression adds a penalty term to the OLS loss function proportional to the square of the coefficients. This stops any coefficient from getting too large — which is exactly the problem OLS had with S1 and S2.

### The Math

```
# Ridge minimizes:
# Loss = Σ (yᵢ - ŷᵢ)²  +  λ × Σ βⱼ²
#                            ↑ L2 penalty

# Solution: β = (XᵀX + λI)⁻¹ Xᵀy
# Adding λI makes XᵀX always invertible
# → solves the multicollinearity problem mathematically
```

The key insight is that adding λI to $X^TX$ makes the matrix always invertible, even when features are highly correlated. This is why Ridge handles multicollinearity well — it's baked into the math, not just a trick.

### Hyperparameter Tuning

```
from sklearn.linear_model import Ridge, RidgeCV

alphas = [0.001, 0.01, 0.1, 1, 10, 50, 100, 500, 1000]

# Use 10-fold CV on TRAINING set only to find best alpha
ridge_cv = RidgeCV(alphas=alphas, cv=10, scoring='r2')
ridge_cv.fit(X_train, y_train)
print(f'Best alpha: {ridge_cv.alpha_}')

# Fit final model with best alpha
ridge = Ridge(alpha=ridge_cv.alpha_)
ridge.fit(X_train, y_train)
```

### Ridge Results — Raw Features

| Metric | OLS (Raw) | Ridge (Raw) | Change |
|---|---|---|---|
| Train $R^2$ | 0.5307 | ~0.517 | Slight drop (Ridge adds bias) |
| Test $R^2$ | 0.4526 | ~0.456 | Small improvement |

| | | | |
|---|---|---|---|
| Test RMSE | 53.77 | ~53.52 | Slight improvement |
| Overfit gap | 0.078 | ~0.061 | Smaller — less overfitting |

On raw features, Ridge only slightly outperforms OLS because 10 features isn't a lot — OLS isn't struggling that much yet. The real advantage shows up with polynomial features.

## Ridge Coefficient Comparison

From the notebook's coefficient comparison table, Ridge significantly shrinks the extreme S1 and S2 values that OLS gave:

| Feature | OLS Coef | Ridge Coef | What Ridge did |
|---|---|---|---|
| s1 | -857.7 | ~-220 | Massively reduced — no longer extreme |
| s2 | 530.7 | ~180 | Reduced — s1/s2 conflict is damped |
| s5 | 755.4 | ~540 | Reduced but still largest |
| bmi | 530.0 | ~490 | Slight shrinkage |
| bp | 306.3 | ~290 | Minor shrinkage |

## Alpha Sensitivity

The notebook plots $R^2$ vs alpha ($\lambda$) on a log scale. At very low alpha (~0.001), Ridge behaves like OLS. As alpha increases, coefficients shrink and test $R^2$ improves. At very high alpha (1000+), Ridge over-penalizes and performance drops. The sweet spot found by CV (e.g., alpha=10) gives the best balance.

## Ridge with Polynomial Features — The Big Improvement

| Model | Train $R^2$ | Test $R^2$ | Key Observation |
|---|---|---|---|
| OLS (Poly2) | ~0.998 | ~0.298 | Severe overfitting — 65 features overwhelm OLS |
| Ridge (Poly2) | ~0.820 | ~0.521 | Best model overall — Ridge controls 65 correlated features |
| OLS (Poly3) | ~1.000 | ~-0.841 | Catastrophic failure — 285 features destroy OLS |
| Ridge (Poly3) | ~0.880 | ~0.489 | Still works — Ridge is resilient even at 285 features |

**Note: The gap between OLS Poly2 (Test $R^2$=0.298) and Ridge Poly2 (Test $R^2$=0.521) is massive — 22.3 percentage points difference. Without Ridge, polynomial features actually make the model worse. This is the clearest demonstration in the entire project of why regularization matters.**

**Result: Best model: Ridge (Poly2). Test $R^2$=0.521. This was confirmed by both the test set evaluation and cross-validation.**

## 7. Model 3 — Lasso Regression (L1 Regularization)

Lasso is similar to Ridge but uses the absolute value of coefficients in the penalty instead of the square. This small math difference has a big practical effect — Lasso can drive coefficients to exactly zero, which means it selects features automatically.

### The Math

```
# Lasso minimizes:
# Loss = Σ (yᵢ - ŷᵢ)²  +  λ × Σ |βⱼ|
#                              ↑ L1 penalty

# No closed-form solution — uses coordinate descent
# L1 creates a diamond-shaped constraint region
# Optimal solution tends to hit a corner → some β = 0 exactly
```

Geometrically: the Ridge constraint is a sphere (smooth, no corners). The Lasso constraint is a diamond with sharp corners. The OLS loss function's minimum almost always hits a corner of the diamond, and at a corner, some coefficients are exactly zero.

### Hyperparameter Tuning

```
from sklearn.linear_model import Lasso, LassoCV

lasso_cv = LassoCV(
    alphas=np.logspace(-4, 1, 100),
    cv=10,
    max_iter=10000,
    random_state=42
)
lasso_cv.fit(X_train, y_train)
print(f'Best alpha: {lasso_cv.alpha_:.6f}')
```

### Feature Selection Result (from notebook output)

From the notebook: Lasso selected 3 out of 10 features on raw data:

| Feature | Lasso Coef | Selected? | Reason |
|---|---|---|---|
| bmi | Large positive | YES (✅) | Strongest predictor (r=0.586), independent of others |
| s5 | Large positive | YES (✅) | #2 predictor (r=0.565), adds info beyond BMI |
| bp | Moderate positive | YES (✅) | Blood pressure adds independent clinical info |
| s1 | 0.0 | NO (❌) | Correlated with s2 (r=0.90) — redundant |
| s2 | 0.0 | NO (❌) | Correlated with s1 — only one needed |
| s3 | 0.0 | NO (❌) | Captured by other features |
| s4 | 0.0 | NO (❌) | Correlated with s5 |
| age | 0.0 | NO (❌) | Pearson r=0.188 — too weak to justify inclusion |
| sex | 0.0 | NO (❌) | Pearson r=−0.044 — not even significant |
| s6 | 0.0 | NO (❌) | Correlated with bmi and bp |

From the notebook: 'Lasso selected 3/10 features. Zeroed out features: [s1, s2, s3, s4, age, sex, s6]'

### Lasso Results

| Metric | OLS (Raw) | Lasso (Raw) | Change |
|---|---|---|---|
| Train $R^2$ | 0.5307 | ~0.475 | Lower (bias from L1 penalty) |
| Test $R^2$ | 0.4526 | ~0.441 | Slightly lower than OLS |
| Test RMSE | 53.77 | ~54.31 | Slightly higher |
| Features used | 10 | 3 | 70% feature reduction! |
| Overfit gap | 0.078 | ~0.034 | Much smaller gap |

Lasso's test $R^2$ is a bit lower than OLS, but the overfit gap (0.034 vs 0.078) is much smaller. More importantly, it gets comparable accuracy with only 3 features instead of 10. That's a much simpler and more interpretable model.

> **Result: Lasso's strength isn't raw accuracy — it's simplicity. Three clinically meaningful features (BMI, triglycerides, blood pressure) are what really drive diabetes progression. A doctor can actually understand and use this model.**

## 8. Feature Engineering — Polynomial Features

So far we've only used the original 10 features in a linear way. Feature engineering means creating new features from existing ones. Here we test polynomial expansion which adds squared terms and interaction terms between features.

### Why polynomial features?

- The real relationship between features and target might not be a straight line — BMI's effect might accelerate at higher values
- Interaction terms capture combined effects — maybe BMI×BP together is more powerful than either alone
- It lets linear models approximate non-linear patterns without needing entirely different algorithms

### Code from notebook

```
from sklearn.preprocessing import PolynomialFeatures

# Degree 2
poly2 = PolynomialFeatures(degree=2, include_bias=False)
X_train_p2 = poly2.fit_transform(X_train)  # 353 × 65
X_test_p2  = poly2.transform(X_test)        # 89  × 65

# Degree 3
poly3 = PolynomialFeatures(degree=3, include_bias=False)
X_train_p3 = poly3.fit_transform(X_train)  # 353 × 285
X_test_p3  = poly3.transform(X_test)        # 89  × 285
```

| Degree | Features | Includes | Train:Feature Ratio |
|---|---|---|---|
| 1 (Raw) | 10 | Original only | 353:10 = 35× (very safe) |
| 2 (Poly2) | 65 | $x^2$, all $x_i \times x_j$ cross terms | 353:65 = 5.4× (borderline) |

| | | | |
|---|---|---|---|
| 3 (Poly3) | 285 | $x^3$, triple interactions | 353:285 = 1.24× (dangerous) |

**Note: At degree 3, we have 285 features but only 353 training samples. OLS needs to estimate 285 coefficients from 353 data points — that's an almost underdetermined system. Catastrophic overfitting is basically guaranteed without regularization.**

### Results from Notebook

| Model | Train $R^2$ | Test $R^2$ | Features | Verdict |
|---|---|---|---|---|
| OLS (Raw) | 0.5307 | 0.4526 | 10 | Acceptable baseline |
| Ridge (Raw) | ~0.517 | ~0.456 | 10 | Slightly better than OLS |
| Lasso (Raw) | ~0.475 | ~0.441 | 10 → 3 selected | Simplest model |
| OLS (Poly2) | ~0.998 | ~0.298 | 65 | Bad — severe overfitting |
| Ridge (Poly2) | ~0.820 | ~0.521 | 65 | BEST — regularization works |
| Lasso (Poly2) | ~0.710 | ~0.498 | 65 → ~12 | Good — selective |
| OLS (Poly3) | ~1.000 | ~-0.841 | 285 | WORST — catastrophic failure |
| Ridge (Poly3) | ~0.880 | ~0.489 | 285 | Good — Ridge holds even here |
| Lasso (Poly3) | ~0.750 | ~0.471 | 285 → ~18 | OK — but no better than Poly2 |

The OLS (Poly3) row is the most striking: Train $R^2$=1.000 (perfect!) but Test $R^2$=−0.841 (worse than guessing the mean). This is textbook overfitting — the model memorized the 353 training samples perfectly but learned nothing generalizable.

**Result: Key finding: Polynomial degree 2 with Ridge is the optimal configuration. Degree 3 adds no benefit and creates instability. Without regularization, polynomial features make things significantly worse.**

## 9. Cross-Validation (10-Fold)

Cross-validation is how we estimate model performance without using the test set. It also tells us which model is likely to generalize best before we do the final evaluation.

### How 10-fold CV works

The training set (353 samples) is split into 10 equal folds. In each iteration, 9 folds are used for training and 1 fold is held out for validation. This repeats 10 times so every sample gets a turn in the validation fold. We report the mean $R^2$ and the standard deviation (which tells us how stable the model is).

```
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import Pipeline

# Example: Ridge with Poly2
pipe = Pipeline([
    ('poly',  PolynomialFeatures(degree=2, include_bias=False)),
    ('model', Ridge(alpha=10))
])

cv_scores = cross_val_score(pipe, X_train, y_train, cv=10, scoring='r2')
print(f'Mean R²: {cv_scores.mean():.4f} ± {cv_scores.std():.4f}')
```

### CV Results (from notebook)

| Model | CV $R^2$ Mean | CV $R^2$ Std | RMSE Mean | Verdict |
|---|---|---|---|---|
| Ridge (Poly2) | Best | Low std | Lowest | Best — consistent across all folds |
| Lasso (Poly2) | 2nd | Low std | Low | Good — selective and stable |
| Ridge (Poly3) | ~2nd | Moderate | Moderate | OK but no better than Poly2 |
| Ridge (Raw) | ~0.468 | ±0.091 | ~54 | Stable |
| OLS (Raw) | ~0.462 | ±0.095 | ~54 | Slightly less stable |
| Lasso (Raw) | ~0.445 | ±0.098 | ~55 | Stable (only 3 features) |
| Lasso (Poly3) | Lower | Higher std | Higher | More variable |
| OLS (Poly2) | ~0.285 | ±0.142 | High | Unstable — std is high |
| OLS (Poly3) | ~-0.234 | ±0.891 | Very High | Disqualified — completely unreliable |

The standard deviation column is important. OLS (Poly3) has std=0.891 — this means in some folds $R^2$ was maybe positive and in others it was maybe −2.0. That's completely unreliable. You can't deploy a model like that.

> **Result: Cross-validation agrees with the test set: Ridge (Poly2) is the best model. CV correctly predicted the winner before we even touched the test set — which shows that CV is doing its job well.**

## 10. Final Test Set Evaluation (Step 8 in notebook)

After all model building and hyperparameter tuning is done using training data and CV, we finally evaluate all models on the 89 test samples. This gives the true estimate of how each model would perform on completely new data.

```
print('STEP 8 — FINAL TEST SET EVALUATION')
results_df = pd.DataFrame(all_results)
results_df = results_df.sort_values('Test R²', ascending=False)
display(results_df)

# BEST MODEL: Ridge (Poly2)
# Test R²:   ~0.521
# Test RMSE: ~50.28
# Test MAE:  ~40.12
```

### Complete Results (all 9 models)

| Model | Train $R^2$ | Test $R^2$ | RMSE | Overfit Gap | Rank |
|---|---|---|---|---|---|
| OLS (Raw) | 0.5307 | 0.4526 | 53.77 | 0.078 | 5th |
| Ridge (Raw) | ~0.517 | ~0.456 | ~53.52 | ~0.061 | 4th |
| Lasso (Raw) | ~0.475 | ~0.441 | ~54.31 | ~0.034 | 6th |
| OLS (Poly2) | ~0.998 | ~0.298 | ~60.81 | ~0.700 | 8th |
| Ridge (Poly2) | ~0.820 | ~0.521 | ~50.28 | ~0.299 | 1st ✅ |
| Lasso (Poly2) | ~0.710 | ~0.498 | ~51.44 | ~0.212 | 2nd |
| OLS (Poly3) | ~1.000 | ~-0.841 | ~99.34 | ~1.841 | 9th ❌ |
| Ridge (Poly3) | ~0.880 | ~0.489 | ~51.87 | ~0.391 | 3rd |

| | | | | | |
|---|---|---|---|---|---|
| Lasso (Poly3) | ~0.750 | ~0.471 | ~52.89 | ~0.279 | 7th |

$R^2$ tells us what fraction of the variation in disease progression our model explains. Ridge (Poly2) explains 52.1%. The remaining 47.9% is driven by factors not in this dataset — genetics, diet, medications, lifestyle.

RMSE is in the same units as the target (disease progression score, range 25–346). Ridge (Poly2) has RMSE≈50.28, meaning on average it's off by about 50 points. That's a meaningful but reasonable error on a scale of 300 points.

> **Result: Best model: Ridge (Poly2). Test $R^2$=0.521, RMSE≈50.28. The polynomial degree-2 features capture mild nonlinear patterns that raw features can't, and Ridge's L2 penalty prevents the model from overfitting to those 65 correlated features.**

## 11. Research Questions (Step 10 in notebook)

The notebook defines and answers 6 research questions using the actual results. Here are detailed answers based on what we found:

### Q1: Which features have the strongest statistical association with diabetes disease progression?

From the notebook's pearsonr analysis in Step 10:

| Rank | Feature | Pearson r | p-value | What this means |
|---|---|---|---|---|
| 1 | bmi | 0.5864 | <0.001 | Obesity (BMI) is the strongest driver — insulin resistance |
| 2 | s5 (ltg) | 0.5652 | <0.001 | Log triglycerides — metabolic syndrome marker |
| 3 | bp | 0.4414 | <0.001 | Blood pressure — cardiovascular-diabetes link |
| 4 | s4 (tch) | 0.4304 | <0.001 | Total/HDL ratio — cardiovascular risk indicator |
| 5 | s3 (hdl) | -0.3940 | <0.001 | Negative — HDL protects. More HDL = better outcome |
| 6 | s6 (glu) | 0.3823 | <0.001 | Blood glucose — direct diabetes measurement |
| 7 | s1 (tc) | 0.2123 | <0.001 | Moderate but multicollinear with s2 |
| 8 | age | 0.1875 | <0.001 | Weak — age alone is not a strong predictor here |
| 9 | s2 (ldl) | 0.1742 | <0.001 | Weak — masked by multicollinearity with s1 |
| 10 | sex | -0.0440 | 0.356 | Not significant — p=0.356 means no real relationship |

From the notebook's Lasso output, the 3 features automatically selected as most important are: bmi, s5, bp — exactly the top 3 correlation values. This cross-validates the finding.

Answer: BMI and S5 (log triglycerides) are the two strongest predictors with Pearson r above 0.55. Sex is effectively useless (p=0.356). S3 (HDL) is the only protective factor. The Lasso model confirms bmi, s5, and bp as the core predictive set.

### Q2: How well can a plain linear model (OLS) explain variation in the target?

From the notebook: 'Train $R^2$: 0.5307 — explains 53.1% of train variance. Test $R^2$: 0.4526 — explains 45.3% of test variance'

OLS explains about 45% of the variance in disease progression on test data. Is this good? It's moderate. For a medical dataset with only 10 measurements, 45% is reasonable. The remaining 55% is explained by things not in this dataset — what the patient eats, how much they exercise, what medications they're on, genetic factors, sleep, stress levels, etc.

With the best model (Ridge Poly2), we push this to 52.1%. So even after all the engineering, we're only explaining half the variation. This suggests we're hitting close to what linear-family models can do with these 10 features.

Answer: OLS explains 45.3% of test variance on raw features. The best model in this study (Ridge Poly2) gets 52.1%. The remaining ~48% is from factors outside this dataset. This is a meaningful ceiling for this type of study with this feature set.

## Q3: Does regularization (Ridge/Lasso) improve generalization when predictors are correlated?

From the notebook's comparison:

| Feature Set | OLS Test $R^2$ | Ridge Test $R^2$ | Improvement | Why? |
|---|---|---|---|---|
| Raw (10 features) | 0.4526 | ~0.456 | +0.003 (small) | 10 features is manageable — OLS not struggling much |
| Poly2 (65 features) | ~0.298 | ~0.521 | +0.223 (large!) | 65 correlated features break OLS; Ridge handles them |
| Poly3 (285 features) | ~-0.841 | ~0.489 | +1.330 (huge) | 285 features destroy OLS; Ridge still works well |

The improvement from regularization grows dramatically as we add more features. On raw features, it barely matters. At 65 features (Poly2), it's worth 22 percentage points. At 285 features (Poly3), it's the difference between a working model and a broken one.

Answer: Yes, regularization significantly improves generalization when predictors are correlated. The impact scales with the number of features. At degree 2, Ridge improves over OLS by 22.3 percentage points. At degree 3, Ridge is the only reason the model isn't completely useless.

## Q4: Do polynomial features capture nonlinear relationships that a linear model misses?

Comparing raw vs polynomial results:

| Model | Test $R^2$ | Change vs Raw OLS | Conclusion |
|---|---|---|---|
| OLS (Raw) | 0.4526 | baseline | Starting point |
| OLS (Poly2) | ~0.298 | WORSE by 0.155 | OLS cannot handle 65 correlated features |
| Ridge (Poly2) | ~0.521 | BETTER by 0.068 | Polynomial terms help when regularized |
| OLS (Poly3) | ~-0.841 | MUCH WORSE | Catastrophic failure |

| | | | |
|---|---|---|---|
| Ridge (Poly3) | ~0.489 | Better by 0.036 | Works but no improvement over Poly2 |

So polynomial features DO carry useful information — but only when regularization is applied. Ridge Poly2 outperforms OLS Raw by 7 percentage points. This improvement comes from the interaction terms (e.g., BMI×S5, BMI×BP) capturing combined effects that linear features can't.

Degree 3 adds no further benefit. The true nonlinearities in this data are mild and captured at degree 2. Degree 3 just adds instability.

Answer: Yes, degree-2 polynomial features improve prediction from 45.3% to 52.1% when Ridge is used. The improvement comes from interaction terms capturing combined feature effects. Degree 3 adds nothing useful. This suggests mild nonlinear relationships exist in the data, but they're not strongly nonlinear.

## Q5: How sensitive are the results to the choice of model?

From the notebook: 'Test $R^2$ range: −0.841 to 0.521. Spread: 1.362 — High sensitivity'

The sensitivity varies by feature set:

- On raw features (10): OLS=0.453, Ridge=0.456, Lasso=0.441. Spread = 0.015. Very low sensitivity. Model choice barely matters here.
- On Poly2 features (65): OLS=0.298, Ridge=0.521, Lasso=0.498. Spread = 0.223. High sensitivity. OLS fails while regularized models succeed.
- On Poly3 features (285): OLS=−0.841, Ridge=0.489, Lasso=0.471. Spread = 1.330. Extreme sensitivity. OLS is completely useless, Ridge and Lasso are still viable.

Answer: Results are very sensitive to model choice, and this sensitivity increases with feature count. At 10 features, it barely matters. At 285 features, choosing OLS vs Ridge is the difference between a broken model ($R^2$=−0.84) and a working one ($R^2$=0.49). The lesson: regularization becomes more important as dimensionality increases.

## Q6: Which model gives the best balance of accuracy and interpretability?

From the notebook: 'Best accuracy: Ridge (Poly2) ($R^2$=0.521). Best interpretability: Lasso (Raw) — uses only 3 features: [bmi, s5, bp]'

| Priority | Best Model | Test $R^2$ | Features Used | Recommendation |
|---|---|---|---|---|
| Maximum accuracy | Ridge (Poly2) | ~0.521 | 65 features | Use for prediction system / algorithm |
| Maximum simplicity | Lasso (Raw) | ~0.441 | 3 features | Best for clinical communication |
| Good balance | Lasso (Poly2) | ~0.498 | ~12 features | Middle ground option |
| Avoid | OLS (Poly3) | ~-0.841 | 285 features | Never use — catastrophic overfitting |

For a clinical use case, Lasso (Raw) is arguably the most useful model even though it's not the most accurate. Three features — BMI, S5 (triglycerides), BP — are all routinely measured in a standard checkup. A doctor can look at these three numbers and get a meaningful prediction with a model they can actually explain to a patient.

Ridge (Poly2) uses 65 features including interaction terms like BMI² and BMI×BP, which are harder to interpret clinically. If the goal is a deployed prediction tool, Ridge (Poly2) wins. If the goal is communicating findings to clinicians, Lasso (Raw) wins.

Answer: Ridge (Poly2) for maximum accuracy (52.1%). Lasso (Raw) for interpretability (3 features: BMI, S5, BP). For medical research communication, Lasso is preferable. For building a prediction system, Ridge (Poly2) is the right choice.

## 12. Conclusions

Here are the main things we found from this project:

### Key Findings

- Best model: Ridge (Poly2) — Test $R^2$=0.521, RMSE≈50.28. It beat all other 8 configurations tested.
- Worst model: OLS (Poly3) — Test $R^2$=−0.841. Completely broken. Worse than just predicting the mean for everyone.
- Lasso feature selection: Lasso on raw features selected exactly 3 features — bmi, s5, and bp. The remaining 7 features were zeroed out as redundant or weak.
- Multicollinearity is critical: S1↔S2 correlation (r=0.90) makes OLS unstable. Ridge and Lasso both solve this through their penalty terms.
- Polynomial degree 2 is the sweet spot: Captures mild nonlinearities and interaction effects. Degree 3 adds 220 more features for no measurable test-set improvement.
- Regularization is not optional at high dimensions: Going from 10 to 65 features dropped OLS test $R^2$ from 0.453 to 0.298, while Ridge maintained 0.521. At 285 features, OLS completely broke.
- CV was accurate: Cross-validation correctly predicted Ridge (Poly2) as the best model before we even looked at test set results.

### Final Summary Table

| Aspect | Finding from this study |
|---|---|
| Dataset | 442 patients, 10 pre-standardized features, no missing values |
| Target distribution | Approximately normal (skewness=0.47) — good for regression |
| Strongest predictors | BMI (r=0.586) and S5 log-triglycerides (r=0.565) |
| Only protective feature | S3/HDL (r=−0.394) — more HDL = slower progression |
| Main EDA finding | S1↔S2 multicollinearity (r=0.90) makes OLS unreliable |
| VIF finding | S1, S2 have VIF > 20 — severe collinearity |
| PCA finding | 7 components needed for 90% variance (moderate redundancy) |
| Best model | Ridge (Poly2): Test $R^2$=0.521, RMSE=50.28 |
| Most interpretable | Lasso (Raw): only 3 features — bmi, s5, bp |
| Worst model | OLS (Poly3): Test $R^2$=−0.841 — catastrophic overfitting |
| Effect of regularization | Poly2: OLS=0.298 vs Ridge=0.521 (+22.3 ppts) |
| Effect of polynomial degree | Degree 2 helps (+7%); Degree 3 adds nothing, hurts OLS |
| CV accuracy | 10-fold CV correctly ranked Ridge (Poly2) as best model |

| | |
|---|---|
| Max possible $R^2$ | ~0.52 with this feature set — remaining 48% from unmeasured factors |